ÉCOLE POLYTECHNIQUE DE THIÈS

DIC1 – GIT

Bases de données NoSQL

Projet de modélisation MongoDB

Domaine 3 : marketplace de produits artisanaux

Dossier de modélisation (livrables 1 à 5)

Réalisé par

Mohamed Oureychiyou Diallo

Aichatou Mamadou Kebe

Ibrahima Pouye

Boubacar Sidibé

Enseignant : Pr Ibrahima Gueye

Octobre 2026

Sommaire

Introduction **2**

1. Modèle de données commenté **2**

1.1 artisans 2

1.2 produits 2

1.3 historique_prix 3

1.4 acheteurs 3

1.5 commandes 3

1.6 avis 4

2. Justification des choix de modélisation **4**

2.1 Données recopiées 4

2.2 Alternatives écartées 5

2.3 Prise en compte des contraintes métier 5

3. Schéma de validation $jsonSchema **5**

4. Design patterns **7**

4.1 Computed : la note globale de l'artisan 7

4.2 Extended Reference : les articles d'une commande 8

5. Jeu de données de démonstration **9**

Introduction

Ce dossier présente le modèle de données MongoDB que nous proposons pour une plateforme mettant en relation des artisans et des acheteurs. Les sections 1 à 5 correspondent aux livrables 1 à 5 du projet : le modèle de données commenté, la justification des choix de modélisation, le schéma de validation, deux design patterns et le jeu de données de démonstration. Les réponses aux quatre questions fondamentales (livrable 6) font l'objet d'un document séparé.

Deux fichiers exécutables dans `mongosh` accompagnent ce dossier : `validation.js`, qui crée les collections et leurs règles de validation, et `donnees.js`, qui insère le jeu de données.

1. Modèle de données commenté

Le modèle comporte six collections : `artisans`, `produits`, `historique_prix`, `acheteurs`, `commandes` et `avis`. Les variantes d'un produit, les articles d'une commande et les favoris d'un acheteur sont stockés dans le document parent ; les autres liens sont représentés par des références, c'est-à-dire des champs suffixés `_id` de type `ObjectId`. Les montants sont exprimés en francs CFA et stockés sous forme d'entiers.

1.1 artisans

Champ	Type	Description
`_id`	ObjectId	Identifiant de l'artisan
`nom`	string	Nom de l'artisan
`nom_boutique`	string	Nom de sa boutique (un artisan possède une boutique)
`email`, `telephone`	string	Coordonnées
`localisation`	objet	Ville et pays de l'atelier (`ville`, `pays`)
`categories`	tableau de string	Catégories d'activité (textile, bijoux, cuir…)
`certifie`	bool	Indique si l'artisan est certifié
`note_globale`	double	Moyenne des notes de ses avis, mise à jour à chaque nouvel avis ; 0 tant qu'il n'en a reçu aucun
`nb_avis`	int	Nombre d'avis reçus, nécessaire à la mise à jour de la moyenne

1.2 produits

Champ	Type	Description
`_id`	ObjectId	Identifiant du produit
`artisan_id`	ObjectId	Référence vers l'artisan qui vend le produit
`titre`, `description`	string	Présentation du produit
`prix`	int	Prix actuel en FCFA
`categories`	tableau de string	Catégories du produit
`photos`	tableau de string	Adresses (URL) des images
`variantes`	tableau d'objets	Déclinaisons du produit, de 1 à 50
`variantes.code`	string	Identifiant de la variante, unique au sein du produit (par exemple `42-NOIR`)
`variantes.taille`, `couleur`, `matiere`	string	Caractéristiques, renseignées seulement lorsqu'elles s'appliquent
`variantes.stock`	int	Quantité disponible pour cette variante

Le stock est porté par chaque variante, puisque c'est une variante précise qui est achetée. Un produit sans déclinaison possède une variante unique.

1.3 historique_prix

Champ	Type	Description
`_id`	ObjectId	Identifiant de l'entrée
`produit_id`	ObjectId	Référence vers le produit concerné
`ancien_prix`, `nouveau_prix`	int	Prix avant et après la modification
`date_changement`	date	Date de la modification

Une entrée est ajoutée à chaque modification du prix d'un produit ; le prix courant reste stocké dans le produit.

1.4 acheteurs

Champ	Type	Description
`_id`	ObjectId	Identifiant de l'acheteur
`nom`, `email`, `telephone`	string	Profil de l'acheteur
`favoris`	tableau d'ObjectId	Références vers les produits sauvegardés

1.5 commandes

Champ	Type	Description
`_id`	ObjectId	Identifiant de la commande
`acheteur_id`	ObjectId	Référence vers l'acheteur
`date_commande`	date	Date de la commande
`articles`	tableau d'objets	Lignes de la commande
`articles.produit_id`	ObjectId	Référence vers le produit acheté
`articles.artisan_id`	ObjectId	Artisan du produit (copie)
`articles.titre`	string	Titre du produit au moment de l'achat (copie)
`articles.code_variante`	string	Variante achetée (copie)
`articles.prix_unitaire`	int	Prix payé, figé à la date de l'achat (copie)
`articles.quantite`	int	Quantité commandée
`montant_total`	int	Somme des articles, calculée à la création de la commande
`statut`	string	`en_attente`, `payee`, `expediee`, `livree` ou `annulee` ; il s'applique à l'ensemble de la commande
`adresse_livraison`	objet	Adresse saisie lors de la commande (`rue`, `ville`, `pays`)

1.6 avis

Champ	Type	Description
`_id`	ObjectId	Identifiant de l'avis
`produit_id`	ObjectId	Produit évalué
`artisan_id`	ObjectId	Artisan du produit (copie, utilisée pour le calcul de sa note)
`acheteur_id`	ObjectId	Auteur de l'avis
`commande_id`	ObjectId	Commande qui atteste l'achat
`note`	int	Note de 1 à 5
`commentaire`	string	Texte de l'avis
`date_avis`	date	Date de dépôt
`achat_verifie`	bool	Indique que l'achat a été contrôlé lors du dépôt ; permet d'afficher la mention « achat vérifié » sans relire la commande

2. Justification des choix de modélisation

Nous avons appliqué la règle vue en cours : une donnée est intégrée (embedded) dans son document parent lorsqu'elle est lue avec lui et que son nombre est limité ; elle est référencée lorsqu'elle est volumineuse ou croît sans limite, qu'elle est lue séparément ou partagée entre plusieurs documents. Pour une relation 1-N référencée dont le côté N peut devenir très grand, la référence est placée dans le document enfant, de sorte qu'aucun tableau ne grossisse indéfiniment dans le document parent.

Relation	Cardinalité	Choix	Justification
Artisan – Boutique	1-1	Champ `nom_boutique` dans l'artisan	Un artisan possède une seule boutique, réduite à son nom et toujours affichée avec son profil.
Artisan – Produit	1-N	Référence `artisan_id` dans le produit	Un artisan peut publier de nombreux produits, consultés et mis à jour (stock) indépendamment de lui.
Produit – Variante	1-N, au plus 50	Intégration dans le produit	Nombre borné ; les variantes sont toujours affichées avec le produit et n'ont pas d'existence propre.
Produit – Historique des prix	1-N	Collection `historique_prix`, référence `produit_id`	Volume non borné ; consultation rare, séparée de la fiche produit.
Acheteur – Commande	1-N	Référence `acheteur_id` dans la commande	Volume non borné ; une commande est aussi consultée par les artisans concernés.
Commande – Produit	N-N	Tableau `articles` intégré, chaque article référençant son produit	Les lignes d'une commande sont toujours lues avec elle et en nombre limité ; le produit, présent dans de nombreuses commandes, reste référencé.
Produit, Acheteur et Commande – Avis	1-N	Collection `avis` portant `produit_id`, `acheteur_id` et `commande_id`	Volume non borné : intégrés au produit, les avis le feraient croître sans limite. Déposer un avis revient à insérer un seul document.
Acheteur – Produit (favoris)	N-N	Tableau d'identifiants dans l'acheteur	Liste courte, lue avec le profil. Elle n'est pas reproduite côté produit, où elle serait sans limite pour un produit populaire sans jamais être consultée.

2.1 Données recopiées

Le modèle recopie volontairement des données à deux endroits. Dans chaque article de commande, le titre, la variante et le prix payé sont recopiés, car une commande doit refléter la vente telle qu'elle a eu lieu, même si le produit est modifié par la suite. L'`artisan_id` y est également recopié : une commande peut regrouper des produits de plusieurs artisans, et chacun doit pouvoir retrouver ses ventes sans jointure. Dans chaque avis, l'`artisan_id` permet de calculer la note d'un artisan à partir de la seule collection `avis`. Ces valeurs ne sont plus modifiées après leur écriture : elles n'ont donc pas à être synchronisées avec le produit.

2.2 Alternatives écartées

Nous avons écarté une liste des produits dans le document de l'artisan : elle serait redondante avec `produits.artisan_id` et sans limite de taille. Nous n'avons pas non plus recopié la note de l'artisan dans ses produits, car elle change à chaque avis et imposerait de mettre à jour tous ses produits. Enfin, les favoris ne justifient pas une collection dédiée, puisque le lien entre un acheteur et un produit ne porte aucune information propre.

2.3 Prise en compte des contraintes métier

Contrainte	Réponse du modèle
Un produit peut avoir jusqu'à 50 variantes	Tableau `variantes` intégré au produit, limité à 50 éléments par le schéma de validation
La note globale d'un artisan est lue très fréquemment	`note_globale` et `nb_avis` stockés dans l'artisan et mis à jour à chaque avis (pattern Computed, section 4.1)
Un avis n'est possible qu'après un achat réel	L'avis référence la commande (`commande_id`). Lors du dépôt, l'application vérifie que la commande existe, appartient à l'acheteur, contient le produit et n'est pas annulée ; sinon l'avis est refusé. Le contrôle réussi est enregistré dans `achat_verifie`.
L'historique des prix doit être conservé	Collection `historique_prix`

3. Schéma de validation $jsonSchema

Le fichier `validation.js` crée les six collections avec un validateur `$jsonSchema`, en mode strict (`validationLevel: "strict"`, `validationAction: "error"`) : toute insertion ou modification non conforme est refusée. Le sujet en demande au moins deux ; nous avons validé les six collections afin de garantir la cohérence de l'ensemble.

Chaque schéma liste les champs obligatoires (`required`), fixe le type de chaque champ (`bsonType`) et refuse les champs non prévus par le modèle (`additionalProperties: false`). Les contraintes du modèle s'y traduisent ainsi : entre 1 et 50 variantes par produit, un stock positif ou nul, des prix, quantités et montants au moins égaux à 1, un statut de commande limité à cinq valeurs, une note d'avis entière comprise entre 1 et 5, et une note globale comprise entre 0 et 5, de type `int` ou `double`. Ce dernier point tient au fait qu'une valeur ronde, comme 4, saisie depuis `mongosh` est enregistrée sous forme d'entier.

Les schémas des collections `produits`, `commandes` et `avis`, qui portent l'essentiel des contraintes, sont reproduits ci-dessous ; ceux des trois autres collections suivent le même principe et figurent dans `validation.js`.

```
const schemaProduits = {
  bsonType: "object",
  required: ["artisan_id", "titre", "description", "prix", "categories", "photos", "variantes"],
  additionalProperties: false,
  properties: {
    _id:         { bsonType: "objectId" },
    artisan_id:  { bsonType: "objectId" },
    titre:       { bsonType: "string" },
    description: { bsonType: "string" },
    prix:        { bsonType: "int", minimum: 1 },
    categories:  { bsonType: "array", items: { bsonType: "string" } },
    photos:      { bsonType: "array", items: { bsonType: "string" } },
    variantes: {
      bsonType: "array", minItems: 1, maxItems: 50,
      items: {
        bsonType: "object",
        required: ["code", "stock"],
        additionalProperties: false,
        properties: {
          code:    { bsonType: "string" },
          taille:  { bsonType: "string" },
          couleur: { bsonType: "string" },
          matiere: { bsonType: "string" },
          stock:   { bsonType: "int", minimum: 0 }
        }
      }
    }
  }
};
```

```
const schemaCommandes = {
  bsonType: "object",
  required: ["acheteur_id", "date_commande", "articles", "montant_total", "statut", "adresse_livraison"],
  additionalProperties: false,
  properties: {
    _id:           { bsonType: "objectId" },
    acheteur_id:   { bsonType: "objectId" },
    date_commande: { bsonType: "date" },
    articles: {
      bsonType: "array", minItems: 1,
      items: {
        bsonType: "object",
        required: ["produit_id", "artisan_id", "titre", "code_variante", "prix_unitaire", "quantite"],
        additionalProperties: false,
        properties: {
          produit_id:    { bsonType: "objectId" },
          artisan_id:    { bsonType: "objectId" },
          titre:         { bsonType: "string" },
          code_variante: { bsonType: "string" },
          prix_unitaire: { bsonType: "int", minimum: 1 },
          quantite:      { bsonType: "int", minimum: 1 }
        }
      }
    },
    montant_total: { bsonType: "int", minimum: 1 },
    statut: { bsonType: "string", enum: ["en_attente", "payee", "expediee", "livree", "annulee"] },
    adresse_livraison: {
      bsonType: "object",
      required: ["rue", "ville", "pays"],
      additionalProperties: false,
      properties: {
        rue:   { bsonType: "string" },
        ville: { bsonType: "string" },
        pays:  { bsonType: "string" }
      }
    }
  }
};
```

```
const schemaAvis = {
  bsonType: "object",
  required: ["produit_id", "artisan_id", "acheteur_id", "commande_id",
             "note", "commentaire", "date_avis", "achat_verifie"],
  additionalProperties: false,
  properties: {
    _id:           { bsonType: "objectId" },
    produit_id:    { bsonType: "objectId" },
    artisan_id:    { bsonType: "objectId" },
    acheteur_id:   { bsonType: "objectId" },
    commande_id:   { bsonType: "objectId" },
    note:          { bsonType: "int", minimum: 1, maximum: 5 },
    commentaire:   { bsonType: "string" },
    date_avis:     { bsonType: "date" },
    achat_verifie: { bsonType: "bool" }
  }
};
```

Chaque collection est créée de la même manière, par exemple :

```
db.createCollection("avis", {
  validator: { $jsonSchema: schemaAvis },
  validationLevel: "strict",
  validationAction: "error"
});
```

Le fichier se termine par neuf tests : chacun tente d'insérer un document volontairement invalide et vérifie qu'il est refusé.

N°	Collection	Document testé	Règle contrôlée
1	`avis`	note égale à 7	Note comprise entre 1 et 5
2	`avis`	note égale à 0	Note comprise entre 1 et 5
3	`commandes`	statut « en_cours »	Statut limité aux cinq valeurs prévues
4	`commandes`	montant total de -100	Montant au moins égal à 1
5	`produits`	variante avec un stock de -3	Stock positif ou nul
6	`produits`	aucune variante	Entre 1 et 50 variantes
7	`produits`	champ `artisan_id` absent	Champ obligatoire
8	`artisans`	note globale « bon »	Type `int` ou `double`
9	`artisans`	note globale égale à 6	Note globale comprise entre 0 et 5

Le validateur ne contrôle qu'un document à la fois : il ne peut pas vérifier que la commande citée dans un avis existe et contient le produit évalué. Ce contrôle relève de l'application, au moment du dépôt de l'avis. Pour modifier les règles d'une collection déjà remplie, on utilise `collMod` ; un exemple avec le niveau `moderate` figure en commentaire dans `validation.js`.

4. Design patterns

4.1 Computed : la note globale de l'artisan

La note globale d'un artisan est la moyenne des notes de tous ses avis. Elle est affichée sur sa boutique et sur chacun de ses produits, et donc lue bien plus souvent qu'un avis n'est déposé. La recalculer à chaque lecture imposerait de parcourir tous ses avis, dont le nombre augmente sans cesse.

Nous appliquons donc le pattern Computed : le calcul est effectué à l'écriture. Le document de l'artisan stocke la moyenne (`note_globale`) et le nombre d'avis (`nb_avis`). Lors du dépôt d'un avis de note *n*, ces deux champs sont mis à jour sans relire les avis existants :

$$\texttt{note_globale} = (\texttt{note_globale} \times \texttt{nb_avis} + n) \,/\, (\texttt{nb_avis} + 1) \quad \text{et} \quad \texttt{nb_avis} = \texttt{nb_avis} + 1$$

Dans notre jeu de données, Fatou Ndiaye a reçu deux avis, notés 5 et 4 :

```
{
  _id: ObjectId("64f000000000000000000001"),
  nom: "Fatou Ndiaye",
  nom_boutique: "Atelier Niani",
  email: "fatou.ndiaye@example.com",
  telephone: "+221770000101",
  localisation: { ville: "Dakar", pays: "Senegal" },
  categories: ["textile", "accessoires"],
  certifie: true,
  note_globale: 4.5,
  nb_avis: 2
}
```

Si elle reçoit un troisième avis noté 3, sa note devient (4,5 × 2 + 3) / 3 = 4,0 et `nb_avis` passe à 3. La contrepartie est une écriture supplémentaire sur le document de l'artisan à chaque avis, ce qui reste négligeable au regard du nombre de lectures. La moyenne est stockée sans arrondi ; l'arrondi n'intervient qu'à l'affichage.

4.2 Extended Reference : les articles d'une commande

Une commande référence les produits achetés. Si chaque article ne contenait que `produit_id`, l'affichage d'une commande nécessiterait une jointure (`$lookup`) sur la collection `produits`, et l'on ne disposerait que du prix actuel des produits, alors que la commande doit conserver le prix effectivement payé.

Nous appliquons le pattern Extended Reference : chaque article conserve la référence `produit_id` et recopie les champs utiles à l'affichage et au suivi des ventes, à savoir `titre`, `code_variante`, `prix_unitaire` et `artisan_id`. Voici la commande passée par Awa Diop le 15 janvier 2026, tirée du jeu de données :

```
{
  _id: ObjectId("64f000000000000000000301"),
  acheteur_id: ObjectId("64f000000000000000000201"),
  date_commande: ISODate("2026-01-15T11:20:00Z"),
  articles: [
    {
      produit_id: ObjectId("64f000000000000000000101"),
      artisan_id: ObjectId("64f000000000000000000001"),
      titre: "Pochette bogolan Niani",
      code_variante: "UNIQUE-BRUN",
      prix_unitaire: 14000,
      quantite: 2
    },
    {
      produit_id: ObjectId("64f000000000000000000103"),
      artisan_id: ObjectId("64f000000000000000000002"),
      titre: "Mini tambour decoratif",
      code_variante: "TAMBOUR-UNIQUE",
      prix_unitaire: 45000,
      quantite: 1
    }
  ],
  montant_total: 73000,
  statut: "livree",
  adresse_livraison: { rue: "12 rue de la Medina", ville: "Dakar", pays: "Senegal" }
}
```

Le prix de la pochette est passé de 14 000 à 18 000 FCFA le 1er mars 2026, comme l'enregistre `historique_prix`, mais la commande conserve les 14 000 FCFA payés. Elle regroupe en outre des produits de deux artisans, Fatou Ndiaye et Amadou Sarr, qui retrouvent chacun leur vente grâce à `articles.artisan_id`. Les champs recopiés sont figés au moment de l'achat ou ne changent jamais, comme l'artisan d'un produit ; la description, les photos et le stock ne sont pas recopiés.

5. Jeu de données de démonstration

Le fichier `donnees.js` insère 32 documents répartis sur les six collections : 4 artisans établis à Dakar, Thiès, Mbour et Kaolack, 8 produits, 3 entrées d'historique de prix, 5 acheteurs, 6 commandes et 6 avis. Les références sont cohérentes : chaque avis porte sur un produit acheté dans une commande non annulée de son auteur, la note globale de chaque artisan correspond à la moyenne de ses avis, et le montant de chaque commande à la somme de ses articles.

Le jeu de données illustre les choix du modèle : les commandes 301 et 303 regroupent des produits de plusieurs artisans ; passées avant une hausse de prix, elles conservent le prix payé, inférieur au prix actuel ; la commande 306, annulée, n'est rattachée à aucun avis. Le script contrôle la cohérence de ces données avant de les insérer et peut être relancé sans créer de doublons.

Les deux fichiers s'exécutent dans `mongosh`, les règles de validation en premier :

```
use marketplace
load("validation.js")
load("donnees.js")
```